

```
In [24]: from __future__ import print_function
import random
import numpy as np
import torch
import torch.nn as nn
import torch.nn.functional as F
import torch.optim as optim
import torchvision
from torch.utils.data import Dataset, DataLoader
from torchvision import datasets
from torchvision import transforms as T
from torch.optim.lr_scheduler import StepLR
import matplotlib.pyplot as plt
```

```
In [25]: class SiameseNetwork(nn.Module):
    def __init__(self):
        super(SiameseNetwork, self).__init__()
        self.resnet = torchvision.models.resnet18(weights=None)
        self.resnet.conv1 = nn.Conv2d(1, 64, kernel_size=(7, 7), stride=(2, 2), padding=3)
        self.fc_in_features = self.resnet.fc.in_features
        self.resnet = torch.nn.Sequential(*(list(self.resnet.children())[:-1]))

        self.fc = nn.Sequential(
            nn.Linear(self.fc_in_features * 2, 256),
            nn.ReLU(inplace=True),
            nn.Linear(256, 1),
        )

        self.sigmoid = nn.Sigmoid()
        self.resnet.apply(self.init_weights)
        self.fc.apply(self.init_weights)

    def init_weights(self, m):
        if isinstance(m, nn.Linear):
            torch.nn.init.xavier_uniform_(m.weight)
            if m.bias is not None:
                m.bias.data.fill_(0.01)

    def forward_once(self, x):
        output = self.resnet(x)
        output = output.view(output.size()[0], -1)
        return output

    def forward(self, input1, input2):
        output1 = self.forward_once(input1)
        output2 = self.forward_once(input2)
        output = torch.cat((output1, output2), 1)
        output = self.fc(output)
        output = self.sigmoid(output)
        return output
```

```
In [26]: class APP_MATCHER(Dataset):
    def __init__(self, root, train, download=False):
        super(APP_MATCHER, self).__init__()
```

```

self.dataset = datasets.MNIST(root, train=train, download=download)
self.data = self.dataset.data.unsqueeze(1).clone()
self.group_examples()

def group_examples(self):
    np_arr = np.array(self.dataset.targets.clone(), dtype=None, copy=True)
    self.grouped_examples = {}
    for i in range(0, 10):
        self.grouped_examples[i] = np.where((np_arr == i))[0]

def __len__(self):
    return self.data.shape[0]

def __getitem__(self, index):
    selected_class = random.randint(0, 9)
    random_index_1 = random.randint(0, self.grouped_examples[selected_class].shape[0])
    index_1 = self.grouped_examples[selected_class][random_index_1]
    image_1 = self.data[index_1].clone().float() / 255.0 # Normalize

    if index % 2 == 0:
        random_index_2 = random.randint(0, self.grouped_examples[selected_class].shape[0])
        while random_index_2 == random_index_1:
            random_index_2 = random.randint(0, self.grouped_examples[selected_class].shape[0])
        index_2 = self.grouped_examples[selected_class][random_index_2]
        image_2 = self.data[index_2].clone().float() / 255.0
        target = torch.tensor(1.0, dtype=torch.float)
    else:
        other_selected_class = random.randint(0, 9)
        while other_selected_class == selected_class:
            other_selected_class = random.randint(0, 9)
        random_index_2 = random.randint(0, self.grouped_examples[other_selected_class].shape[0])
        index_2 = self.grouped_examples[other_selected_class][random_index_2]
        image_2 = self.data[index_2].clone().float() / 255.0
        target = torch.tensor(0.0, dtype=torch.float)

    return image_1, image_2, target

```

```

In [27]: def train(model, device, train_loader, optimizer, epoch):
    model.train()
    criterion = nn.BCELoss()

    for batch_idx, (images_1, images_2, targets) in enumerate(train_loader):
        images_1, images_2, targets = images_1.to(device), images_2.to(device), targets.to(device)
        optimizer.zero_grad()
        outputs = model(images_1, images_2).squeeze()
        loss = criterion(outputs, targets)
        loss.backward()
        optimizer.step()

        if batch_idx % 50 == 0:
            print(f'Train Epoch: {epoch} [{batch_idx * len(images_1)} / {len(train_loader)}] \t Loss: {loss.item():.4f}')

In [28]: def test(model, device, test_loader):
    model.eval()

```

```

test_loss = 0
correct = 0
criterion = nn.BCELoss()

with torch.no_grad():
    for images_1, images_2, targets in test_loader:
        images_1, images_2, targets = images_1.to(device), images_2.to(device),
        outputs = model(images_1, images_2).squeeze()
        test_loss += criterion(outputs, targets).item()
        pred = (outputs > 0.5).float()
        correct += pred.eq(targets).sum().item()

test_loss /= len(test_loader.dataset)
accuracy = 100. * correct / len(test_loader.dataset)

print(f'\nTest set: Average loss: {test_loss:.4f}, Accuracy: {correct}/{len(test_loader.dataset)}')

```

```

In [29]: class Args:
    batch_size = 64
    test_batch_size = 1000
    epochs = 5 # 减少 epoch 数以加快演示速度
    lr = 1.0
    gamma = 0.7
    log_interval = 50
    seed = 1
    save_model = True
    dry_run = False
    no_accel = False # 设置 True 强制使用 CPU

# 初始化
args = Args()
device = torch.device("cuda" if torch.cuda.is_available() and not args.no_accel else "cpu")
print(f"Using device: {device}")

torch.manual_seed(args.seed)

```

Using device: cuda

Out[29]: <torch._C.Generator at 0x1f2127186f0>

```

In [30]: # 创建数据加载器
train_dataset = APP_MATCHER('./data', train=True, download=True)
test_dataset = APP_MATCHER('./data', train=False, download=True)
train_loader = DataLoader(train_dataset, batch_size=args.batch_size, shuffle=True)
test_loader = DataLoader(test_dataset, batch_size=args.test_batch_size)

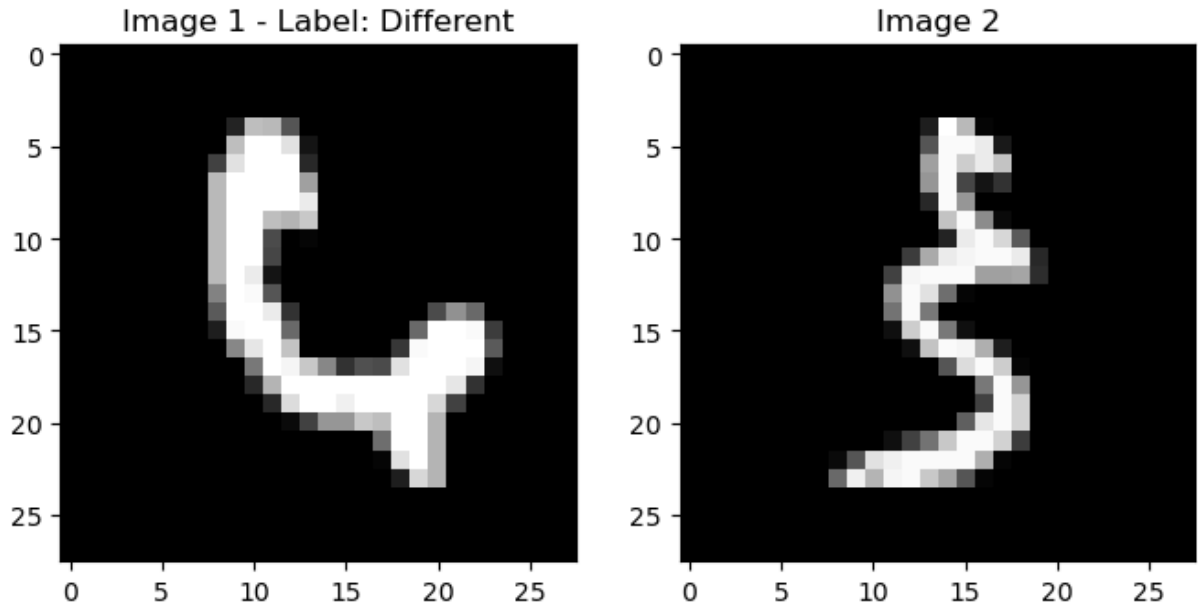
# 可视化样本
sample_idx = random.randint(0, len(train_dataset)-1)
img1, img2, label = train_dataset[sample_idx]

plt.figure(figsize=(8, 4))
plt.subplot(1, 2, 1)
plt.title(f'Image 1 - Label: {"Same" if label == 1.0 else "Different"}')
plt.imshow(img1.squeeze(), cmap='gray')

plt.subplot(1, 2, 2)

```

```
plt.title('Image 2')
plt.imshow(img2.squeeze(), cmap='gray')
plt.show()
print(f"Sample pair label: {label.item()}")
```



Sample pair label: 0.0

```
In [31]: # 初始化模型和优化器
model = SiameseNetwork().to(device)
optimizer = optim.Adadelta(model.parameters(), lr=args.lr)

# 训练循环
for epoch in range(1, args.epochs + 1):
    train(model, device, train_loader, optimizer, epoch)
    test(model, device, test_loader)

    # 更新学习率
    scheduler = StepLR(optimizer, step_size=1, gamma=args.gamma)
    scheduler.step()

# 保存模型
if args.save_model:
    torch.save(model.state_dict(), "siamese_network.pt")
    print("Model saved to siamese_network.pt")
```

```

Train Epoch: 1 [0/60000 (0%)]    Loss: 0.847080
Train Epoch: 1 [3200/60000 (5%)]    Loss: 0.386364
Train Epoch: 1 [6400/60000 (11%)]    Loss: 0.307154
Train Epoch: 1 [9600/60000 (16%)]    Loss: 0.285577
Train Epoch: 1 [12800/60000 (21%)]    Loss: 0.141334
Train Epoch: 1 [16000/60000 (27%)]    Loss: 0.214384
Train Epoch: 1 [19200/60000 (32%)]    Loss: 0.075546
Train Epoch: 1 [22400/60000 (37%)]    Loss: 0.207708
Train Epoch: 1 [25600/60000 (43%)]    Loss: 0.183802
Train Epoch: 1 [28800/60000 (48%)]    Loss: 0.114281
Train Epoch: 1 [32000/60000 (53%)]    Loss: 0.074724
Train Epoch: 1 [35200/60000 (59%)]    Loss: 0.023166
Train Epoch: 1 [38400/60000 (64%)]    Loss: 0.065200
Train Epoch: 1 [41600/60000 (69%)]    Loss: 0.042335
Train Epoch: 1 [44800/60000 (75%)]    Loss: 0.044663
Train Epoch: 1 [48000/60000 (80%)]    Loss: 0.014825
Train Epoch: 1 [51200/60000 (85%)]    Loss: 0.009960
Train Epoch: 1 [54400/60000 (91%)]    Loss: 0.121489
Train Epoch: 1 [57600/60000 (96%)]    Loss: 0.060250

```

Test set: Average loss: 0.0001, Accuracy: 9783/10000 (97.83%)

```

Train Epoch: 2 [0/60000 (0%)]    Loss: 0.063853

```

```

D:\Anaconda\Lib\site-packages\torch\optim\lr_scheduler.py:182: UserWarning: Detected
call of `lr_scheduler.step()` before `optimizer.step()`. In PyTorch 1.1.0 and later,
you should call them in the opposite order: `optimizer.step()` before `lr_scheduler.
step()`. Failure to do this will result in PyTorch skipping the first value of the
learning rate schedule. See more details at https://pytorch.org/docs/stable/optim.ht
ml#how-to-adjust-learning-rate
warnings.warn(

```

Train Epoch: 2	[3200/60000 (5%)]	Loss: 0.127308
Train Epoch: 2	[6400/60000 (11%)]	Loss: 0.063054
Train Epoch: 2	[9600/60000 (16%)]	Loss: 0.080570
Train Epoch: 2	[12800/60000 (21%)]	Loss: 0.064976
Train Epoch: 2	[16000/60000 (27%)]	Loss: 0.014655
Train Epoch: 2	[19200/60000 (32%)]	Loss: 0.061523
Train Epoch: 2	[22400/60000 (37%)]	Loss: 0.078213
Train Epoch: 2	[25600/60000 (43%)]	Loss: 0.031999
Train Epoch: 2	[28800/60000 (48%)]	Loss: 0.037099
Train Epoch: 2	[32000/60000 (53%)]	Loss: 0.005302
Train Epoch: 2	[35200/60000 (59%)]	Loss: 0.037083
Train Epoch: 2	[38400/60000 (64%)]	Loss: 0.021303
Train Epoch: 2	[41600/60000 (69%)]	Loss: 0.010682
Train Epoch: 2	[44800/60000 (75%)]	Loss: 0.113345
Train Epoch: 2	[48000/60000 (80%)]	Loss: 0.023158
Train Epoch: 2	[51200/60000 (85%)]	Loss: 0.035301
Train Epoch: 2	[54400/60000 (91%)]	Loss: 0.090851
Train Epoch: 2	[57600/60000 (96%)]	Loss: 0.005756

Test set: Average loss: 0.0000, Accuracy: 9890/10000 (98.90%)

Train Epoch: 3	[0/60000 (0%)]	Loss: 0.051573
Train Epoch: 3	[3200/60000 (5%)]	Loss: 0.014949
Train Epoch: 3	[6400/60000 (11%)]	Loss: 0.003203
Train Epoch: 3	[9600/60000 (16%)]	Loss: 0.016717
Train Epoch: 3	[12800/60000 (21%)]	Loss: 0.040620
Train Epoch: 3	[16000/60000 (27%)]	Loss: 0.053419
Train Epoch: 3	[19200/60000 (32%)]	Loss: 0.012437
Train Epoch: 3	[22400/60000 (37%)]	Loss: 0.015851
Train Epoch: 3	[25600/60000 (43%)]	Loss: 0.047846
Train Epoch: 3	[28800/60000 (48%)]	Loss: 0.003285
Train Epoch: 3	[32000/60000 (53%)]	Loss: 0.013177
Train Epoch: 3	[35200/60000 (59%)]	Loss: 0.001245
Train Epoch: 3	[38400/60000 (64%)]	Loss: 0.017602
Train Epoch: 3	[41600/60000 (69%)]	Loss: 0.003448
Train Epoch: 3	[44800/60000 (75%)]	Loss: 0.001625
Train Epoch: 3	[48000/60000 (80%)]	Loss: 0.059379
Train Epoch: 3	[51200/60000 (85%)]	Loss: 0.035166
Train Epoch: 3	[54400/60000 (91%)]	Loss: 0.006073
Train Epoch: 3	[57600/60000 (96%)]	Loss: 0.006262

Test set: Average loss: 0.0000, Accuracy: 9923/10000 (99.23%)

Train Epoch: 4	[0/60000 (0%)]	Loss: 0.000754
Train Epoch: 4	[3200/60000 (5%)]	Loss: 0.030139
Train Epoch: 4	[6400/60000 (11%)]	Loss: 0.093155
Train Epoch: 4	[9600/60000 (16%)]	Loss: 0.015829
Train Epoch: 4	[12800/60000 (21%)]	Loss: 0.012501
Train Epoch: 4	[16000/60000 (27%)]	Loss: 0.000686
Train Epoch: 4	[19200/60000 (32%)]	Loss: 0.019517
Train Epoch: 4	[22400/60000 (37%)]	Loss: 0.012536
Train Epoch: 4	[25600/60000 (43%)]	Loss: 0.009837
Train Epoch: 4	[28800/60000 (48%)]	Loss: 0.004221
Train Epoch: 4	[32000/60000 (53%)]	Loss: 0.025582
Train Epoch: 4	[35200/60000 (59%)]	Loss: 0.005465
Train Epoch: 4	[38400/60000 (64%)]	Loss: 0.000594

```
Train Epoch: 4 [41600/60000 (69%)]      Loss: 0.009642
Train Epoch: 4 [44800/60000 (75%)]      Loss: 0.026164
Train Epoch: 4 [48000/60000 (80%)]      Loss: 0.049939
Train Epoch: 4 [51200/60000 (85%)]      Loss: 0.000623
Train Epoch: 4 [54400/60000 (91%)]      Loss: 0.002098
Train Epoch: 4 [57600/60000 (96%)]      Loss: 0.000703
```

Test set: Average loss: 0.0000, Accuracy: 9917/10000 (99.17%)

```
Train Epoch: 5 [0/60000 (0%)]      Loss: 0.097183
Train Epoch: 5 [3200/60000 (5%)]    Loss: 0.001362
Train Epoch: 5 [6400/60000 (11%)]   Loss: 0.001210
Train Epoch: 5 [9600/60000 (16%)]   Loss: 0.094395
Train Epoch: 5 [12800/60000 (21%)]  Loss: 0.088524
Train Epoch: 5 [16000/60000 (27%)]  Loss: 0.008384
Train Epoch: 5 [19200/60000 (32%)]  Loss: 0.000598
Train Epoch: 5 [22400/60000 (37%)]  Loss: 0.113292
Train Epoch: 5 [25600/60000 (43%)]  Loss: 0.004611
Train Epoch: 5 [28800/60000 (48%)]  Loss: 0.002502
Train Epoch: 5 [32000/60000 (53%)]  Loss: 0.003034
Train Epoch: 5 [35200/60000 (59%)]  Loss: 0.000092
Train Epoch: 5 [38400/60000 (64%)]  Loss: 0.000646
Train Epoch: 5 [41600/60000 (69%)]  Loss: 0.000541
Train Epoch: 5 [44800/60000 (75%)]  Loss: 0.011319
Train Epoch: 5 [48000/60000 (80%)]  Loss: 0.106392
Train Epoch: 5 [51200/60000 (85%)]  Loss: 0.002417
Train Epoch: 5 [54400/60000 (91%)]  Loss: 0.010498
Train Epoch: 5 [57600/60000 (96%)]  Loss: 0.102191
```

Test set: Average loss: 0.0000, Accuracy: 9953/10000 (99.53%)

Model saved to siamese_network.pt

In []: